

λ_p : Paths through Dependent Pairs

Anonymous Author(s)

Abstract

We present a monadic-normal-form calculus in which paths traverse immutable chains of dependent pairs. Pair definitions store either a term reference or a type, and the member signature may refer to the pair's first component. The static semantics distinguishes term singletons from selected abstract type members. If singleton symmetry is instead allowed to treat abstract membership as path identity, a closed, well-typed program becomes stuck. The calculus uses a conservative subtyping rule for dependent pairs. It permits member types to change when the first component has singleton type, and permits the first component to be widened separately. We prove type safety in Lean. The proof records exact value-introduction types in the store and interprets structural subtyping by finite, proof-relevant coercion codes. Every endpoint of a finite execution of a closed, well-typed term is final or can take another step.

Keywords: path-dependent types, dependent pairs, singleton types, type soundness, Lean

1 Introduction

A path-dependent type assigns a type component to a stable term path. The usual object-calculus presentation combines this mechanism with recursive objects, refinements, intersections, and a substantial subtyping relation [1, 5]. The purpose of λ_p is narrower: to isolate paths, abstract type members, and dependency between the two components of a pair.

Values in λ_p are functions and immutable dependent pairs. A pair contains a first component and either a term reference or a type definition. Its member signature may mention the first component. Paths follow first projections and named members through chains of pairs; a missed label resumes the search in the first component. The static and dynamic rules expose this path mechanism directly.

Two design points are essential to the result in this paper. First, the static semantics distinguishes the singleton type $\{p\}$, which states that a term denotes the same value as p , from the abstract type selection $p.A$. This calculus makes the distinction syntactic. An earlier version used one constructor for both roles. We give a closed counterexample to that version: singleton symmetry turns membership in an abstract type into a false path equality.

Second, dependent-pair subtyping is deliberately conservative. One rule widens the first component without changing the member. A second rule changes the member only when the first component is the singleton of a path, and retains an explicit subtyping premise after opening the member at that path. The rules suffice to hide exact type definitions

κ	$::=$	$\star \mid \iota$
p, q	$::=$	$x \mid p.1 \mid p.a$
T, U	$::=$	$\top \mid \perp \mid (x : S) \rightarrow T \mid \langle x : S; a : \tau^\kappa \rangle$ $\mid \{p\} \mid p.A$
$\tau^\star$	$::=$	T
τ^ι	$::=$	$L..U$
$\delta^\star$	$::=$	z
δ^ι	$::=$	T
t, u	$::=$	$p \mid \lambda(x : S).t \mid \langle y; a = \delta^\kappa \rangle$ $\mid p\ q \mid \text{let } x = t \text{ in } u \mid (t : T)$
v	$::=$	$\lambda(x : S).t \mid \langle y; a = \delta^\kappa \rangle$
Γ	$::=$	$\emptyset \mid \Gamma, x : T$

Figure 1. Source syntax. Uppercase labels conventionally denote type members.

behind dependent abstract bounds. We do not know whether the natural unrestricted pair rule is unsound; the restriction records the evidence required by the direct semantic proof.

The main technical result is a Lean-checked finite-run safety theorem. The proof has three parts.

- An exact store invariant records every allocated value at its syntax-directed introduction type.
- A store-indexed path equivalence accounts for paths that resolve to the same location. A proof-only structural typing layer admits conversion by this equivalence.
- A finite syntax of semantic coercions interprets every structural subtyping derivation. Applying a coercion to a location singleton yields the canonical forms used by progress and preservation.

The result concerns a deliberately limited language. There are no recursive objects, recursive allocation, mutation, intersections, unions, or general source terms outside monadic normal form. We prove neither normalization nor decidability, and declarative transitivity remains a primitive rule.

2 The calculus

2.1 Syntax and binding

Figure 1 gives the syntax. Kinds distinguish term members ($\star$) from type members (ι). A generalized type τ^κ is a proper type at kind $\star$, and an interval at kind ι . A definition has the corresponding kind: a term member stores a variable, while a type member stores a proper type.

The binder x in a function scopes over its codomain. In $\langle x : S; a : \tau \rangle$, it scopes over the member signature τ . We write $\tau[p/x]$ for capture-avoiding opening. The mechanization uses intrinsically scoped de Bruijn syntax; the paper uses names.

Terms are in monadic normal form. Pair components are variables, and the operator and operand of an application are

paths. This restriction leaves only **let** frames in the evaluation context and makes allocation explicit.

The four source judgments are

$$\Gamma \vdash p :: \tau^K \quad \Gamma \vdash \tau_1^K <: \tau_2^K \quad \Gamma \vdash \tau^K \text{ wf} \quad \Gamma \vdash t : T.$$

The double colon denotes *precise path typing*: unlike term typing, it does not contain subsumption.

2.2 Paths and pair chains

The precise path rules appear in the upper half of [figure 2](#). A variable synthesizes its context entry. A first projection synthesizes the first-component type. Selecting the matching member opens its signature with the first projection of the receiver.

Pairs also represent record chains. If the outer label is b but the requested label is a , rule P-MISS continues from $p.1.a$. This static rule matches the recursive missed-label case of path resolution in [figure 5](#).

2.3 Subtyping and well-formedness

[Figure 3](#) gives the complete declarative subtyping relation. It includes reflexivity and primitive transitivity, as well as the usual extremal and function rules. Rule S-WIDEN exposes the precise type of a singleton path. Rule S-ALIAS is singleton symmetry: if p has the *term singleton* $\{q\}$, then the singleton subtyping relation may be reversed.

Abstract type selections use the interval synthesized by precise member lookup. The premise $L <: U$ is an explicit nonemptiness guard. It is retained both here and in interval formation and subtyping. Interval subtyping is contravariant in its lower bound and covariant in its upper bound.

The two pair rules separate two operations. S-PAIR-FST changes the first-component type and leaves the member fixed. S-PAIR-SINGLE changes the member under a singleton first component. Its last premise is not a well-formedness condition: it is a second subtyping derivation after opening the member at p . Declarative transitivity composes the two operations. [Section 3](#) explains the restriction and its use in the proof.

The lower half of [figure 2](#) gives all well-formedness rules. Singleton formation requires a term-denoting path, while abstract selection requires a pair with an interval member. Functions and pairs check their dependent component under the corresponding binder. Rule W-BOUNDS repeats the nonemptiness condition.

2.4 Term typing

[Figure 4](#) gives term typing. A path receives its singleton type; S-WIDEN can subsequently expose its precise type. Application is dependent. The two pair-introduction rules assign exact member signatures. A term definition receives $\{z\}$; a type definition receives $T..T$. The first component is also exact, namely $\{y\}$.

The result type of a let expression cannot depend on its bound variable. The side condition is represented in the intrinsically scoped development by weakening T below the binder. Subsumption requires the target to be well-formed.

2.5 Store and machine

A store σ is an immutable sequence of values. A stored value may refer only to earlier cells. We write $\sigma(x) = v$ for lookup and $\sigma, x \mapsto v$ for extension at a fresh location. There is one intrinsically scoped namespace for source variables and run-time locations; the named presentation calls store-bound variables locations when useful.

Source path resolution, written $\sigma \vdash p \Downarrow x$, is big-step. The rules are at the top of [figure 5](#). A variable resolves to itself. Projection returns the stored first component. A matching term member returns its stored location. A missed label continues at the first component. Resolution is deterministic, although it is partial on arbitrary stores and paths.

Machine states are triples $\langle \sigma, K, t \rangle$. Continuations contain only let frames $F = \text{let } x = [] \text{ in } t$. The six transition rules appear at the bottom of [figure 5](#). Application resolves both paths before opening a stored abstraction. A non-variable path normalizes to its resolved location. A returned location is substituted directly into a suspended let body; a syntactic value is instead allocated at a fresh location. Store extension weakens the old continuation, written $K^\uparrow$, into the extended scope. Ascriptions have no dynamic effect.

A state is final when its continuation is empty and its term is either a valid store location or a syntactic value. The initial state of t is $\text{initial}(t) = \langle \emptyset, [], t \rangle$.

3 Singletons, selections, and pair abstraction

3.1 Why singleton types and selections are distinct

The term singleton $\{p\}$ asserts value identity. In particular, S-ALIAS is justified when precise typing says that one path has another path's singleton type. An abstract selection $p.A$, by contrast, denotes an unknown type between the bounds stored at A . Membership in $p.A$ says nothing by itself about path identity.

Some distinction between the two roles is required by a source-level counterexample. In a precursor to λ_p , both forms were represented by a single type constructor and singleton symmetry accepted either role. Let

$$\begin{aligned} f &= \lambda(x : \top).x, \\ q &= \langle f; A = \top \rangle, \\ h &= \lambda(z : q.A).q. \end{aligned}$$

The historical rules type the body of h by the chain

$$\{q\} <: \top <: q.A <: \{z\}.$$

The last edge applies singleton symmetry to the context binding $z : q.A$, treating abstract membership as equality.

$$\begin{array}{c}
\frac{\Gamma(x) = T}{\Gamma \vdash x :: T} \text{P-VAR} \quad \frac{\Gamma \vdash p :: \langle x:S; a:\tau \rangle}{\Gamma \vdash p.1 :: S} \text{P-FST} \quad \frac{\Gamma \vdash p :: \langle x:S; a:\tau \rangle}{\Gamma \vdash p.a :: \tau[p.1/x]} \text{P-SEL} \quad \frac{\Gamma \vdash p :: \langle x:S; b:\tau' \rangle \quad \Gamma \vdash p.1.a :: \tau \quad a \neq b}{\Gamma \vdash p.a :: \tau} \text{P-MISS} \\
\hline
\frac{}{\Gamma \vdash \perp \text{ wf}} \text{W-BOT} \quad \frac{}{\Gamma \vdash \top \text{ wf}} \text{W-TOP} \quad \frac{\Gamma \vdash p :: T}{\Gamma \vdash \{p\} \text{ wf}} \text{W-SINGLE} \quad \frac{\Gamma \vdash p :: \langle x:S; A:L..U \rangle}{\Gamma \vdash p.A \text{ wf}} \text{W-SELECT} \\
\frac{\Gamma \vdash S \text{ wf} \quad \Gamma, x:S \vdash T \text{ wf}}{\Gamma \vdash (x:S) \rightarrow T \text{ wf}} \text{W-FUN} \quad \frac{\Gamma \vdash S \text{ wf} \quad \Gamma, x:S \vdash \tau \text{ wf}}{\Gamma \vdash \langle x:S; a:\tau \rangle \text{ wf}} \text{W-PAIR} \quad \frac{\Gamma \vdash L \text{ wf} \quad \Gamma \vdash U \text{ wf} \quad \Gamma \vdash L <: U}{\Gamma \vdash L..U \text{ wf}} \text{W-BOUNDS}
\end{array}$$

Figure 2. Precise path typing (above) and well-formedness (below).

$$\begin{array}{c}
\frac{}{\Gamma \vdash \tau <: \tau} \text{S-REFL} \quad \frac{\Gamma \vdash \tau_1 <: \tau_2 \quad \Gamma \vdash \tau_2 <: \tau_3}{\Gamma \vdash \tau_1 <: \tau_3} \text{S-TRANS} \quad \frac{}{\Gamma \vdash \perp <: T} \text{S-BOT} \quad \frac{}{\Gamma \vdash T <: \top} \text{S-TOP} \quad \frac{\Gamma \vdash p :: T}{\Gamma \vdash \{p\} <: T} \text{S-WIDEN} \\
\frac{\Gamma \vdash p :: \{q\}}{\Gamma \vdash \{q\} <: \{p\}} \text{S-ALIAS} \quad \frac{\Gamma \vdash p.A :: L..U \quad \Gamma \vdash L <: U}{\Gamma \vdash L <: p.A} \text{S-SEL-LO} \quad \frac{\Gamma \vdash p.A :: L..U \quad \Gamma \vdash L <: U}{\Gamma \vdash p.A <: U} \text{S-SEL-HI} \\
\frac{\Gamma \vdash S' <: S \quad \Gamma, x:S' \vdash T <: T'}{\Gamma \vdash (x:S) \rightarrow T <: (x:S') \rightarrow T'} \text{S-FUN} \quad \frac{\Gamma \vdash S <: S'}{\Gamma \vdash \langle x:S; a:\tau \rangle <: \langle x:S'; a:\tau \rangle} \text{S-PAIR-FST} \\
\frac{\Gamma \vdash p :: P \quad \Gamma, x:\{p\} \vdash \tau <: \tau' \quad \Gamma \vdash \tau[p/x] <: \tau'[p/x]}{\Gamma \vdash \langle x:\{p\}; a:\tau \rangle <: \langle x:\{p\}; a:\tau' \rangle} \text{S-PAIR-SINGLE} \quad \frac{\Gamma \vdash L' <: L \quad \Gamma \vdash U <: U' \quad \Gamma \vdash L <: U}{\Gamma \vdash L..U <: L'..U'} \text{S-BOUNDS}
\end{array}$$

Figure 3. Generalized subtyping. Premises and conclusions have the same kind.

$$\begin{array}{c}
\frac{\Gamma \vdash p :: T}{\Gamma \vdash p : \{p\}} \text{T-PATH} \quad \frac{\Gamma, x:S \vdash t:T \quad \Gamma \vdash S \text{ wf}}{\Gamma \vdash \lambda(x:S).t : (x:S) \rightarrow T} \text{T-ABS} \quad \frac{\Gamma \vdash p : (x:S) \rightarrow T \quad \Gamma \vdash q : S}{\Gamma \vdash p q : T[q/x]} \text{T-APP} \\
\frac{\Gamma(y) = S \quad \Gamma(z) = T}{\Gamma \vdash \langle y; a=z \rangle : \langle x:\{y\}; a:\{z\} \rangle} \text{T-PAIR} \quad \frac{\Gamma(y) = S \quad \Gamma \vdash T \text{ wf}}{\Gamma \vdash \langle y; A=T \rangle : \langle x:\{y\}; A:T..T \rangle} \text{T-TYPE-PAIR} \\
\frac{\Gamma \vdash s:S \quad \Gamma \vdash T \text{ wf} \quad \Gamma, x:S \vdash t:T \quad x \notin \text{fv}(T)}{\Gamma \vdash \text{let } x = s \text{ in } t : T} \text{T-LET} \quad \frac{\Gamma \vdash t:T \quad \Gamma \vdash T \text{ wf}}{\Gamma \vdash (t:T) : T} \text{T-ASCRIIBE} \\
\frac{\Gamma \vdash t:S \quad \Gamma \vdash S <: T \quad \Gamma \vdash T \text{ wf}}{\Gamma \vdash t:T} \text{T-SUB}
\end{array}$$

Figure 4. Term typing. Names bound by pair types are in scope in their member signature.

$$\begin{array}{c}
\frac{}{\sigma \vdash x \Downarrow x} \text{R-VAR} \quad \frac{\sigma \vdash p \Downarrow x \quad \sigma(x) = \langle y; a=\delta \rangle}{\sigma \vdash p.1 \Downarrow y} \text{R-FST} \quad \frac{\sigma \vdash p \Downarrow x \quad \sigma(x) = \langle y; a=z \rangle}{\sigma \vdash p.a \Downarrow z} \text{R-HIT} \\
\frac{\sigma \vdash p \Downarrow x \quad \sigma(x) = \langle y; b=\delta \rangle \quad a \neq b \quad \sigma \vdash y.a \Downarrow z}{\sigma \vdash p.a \Downarrow z} \text{R-MISS} \\
\hline
\frac{\sigma \vdash p \Downarrow f \quad \sigma \vdash q \Downarrow y \quad \sigma(f) = \lambda(x:S).t}{\langle \sigma, K, p q \rangle \longrightarrow \langle \sigma, K, t[y/x] \rangle} \text{E-APP} \quad \frac{\sigma \vdash p \Downarrow x \quad p \text{ not a variable}}{\langle \sigma, K, p \rangle \longrightarrow \langle \sigma, K, x \rangle} \text{E-PATH} \quad \frac{F = \text{let } x = [] \text{ in } t}{\langle \sigma, K, \text{let } x = s \text{ in } t \rangle \longrightarrow \langle \sigma, F :: K, s \rangle} \text{E-LET} \\
\frac{F = \text{let } x = [] \text{ in } t}{\langle \sigma, F :: K, y \rangle \longrightarrow \langle \sigma, K, t[y/x] \rangle} \text{E-RETURN} \quad \frac{F = \text{let } x = [] \text{ in } t \quad v \text{ value} \quad \ell \text{ fresh}}{\langle \sigma, F :: K, v \rangle \longrightarrow \langle \sigma[\ell \mapsto v], K^\uparrow, t[\ell/x] \rangle} \text{E-ALLOC} \quad \frac{}{\langle \sigma, K, (t:T) \rangle \longrightarrow \langle \sigma, K, t \rangle} \text{E-ASCRIIBE}
\end{array}$$

Figure 5. Big-step path resolution (above) and CK-machine transitions (below).

The same rules accept f as an argument of type $q.A$. Thus hf is assigned $\{f\}$, although it returns q .

The closed program

```
let f = λ(x : T).x in
let q = ⟨f; A = T⟩ in
let h = λ(z : q.A).q in
let r = hf in rf
```

takes nine machine steps to an application whose operator location contains the pair q . It is neither final nor reducible. Every interval in the derivation is $\top..T$, so this is not a bad-bounds example. It also does not depend on transitivity being primitive: any ordinary transitivity lemma composes the displayed chain.

The full derivation and trace are retained in the historical library, in module `SourceUnsoundnessCounterexample`. In the present syntax, a binding $z : q.A$ cannot be a premise for `S-ALIAS`. Theorems `historical_body_subtyping_blocked` and `selection_to_argument_singleton_blocked` show that the crucial subtypings remain underivable even with arbitrary uses of primitive transitivity.

The counterexample rules out treating arbitrary abstract membership as singleton evidence. It does not show that two abstract-syntax constructors are the only possible repair: one could instead add a term-denotation classification premise to singleton symmetry. Separate constructors make the distinction explicit and are the design adopted here.

3.2 The pair rule

The unrestricted rule one might first write is

$$\frac{\Gamma \vdash S <: S' \quad \Gamma, x : S \vdash \tau <: \tau'}{\Gamma \vdash \langle x:S; a:\tau \rangle <: \langle x:S'; a:\tau' \rangle} \quad (\text{Unrestricted Pair})$$

Its semantic interpretation must work for an arbitrary run-time first component y that realizes S . In particular, it must turn a realizer of $\tau[y/x]$ into a realizer of $\tau'[y/x]$. The raw subtyping derivation under $x : S$ does not contain a structurally smaller semantic coercion specialized to y . Representing the required inhabitant-indexed family directly as a function would place the realization relation negatively in its own definition.

`S-PAIR-SINGLE` fixes a path p and supplies the specialized subtyping derivation explicitly. If the stored first component y realizes $\{p\}$, then y and p resolve to the same location. Run-time conversion identifies

$$\tau[y/x] \equiv_{\sigma} \tau[p/x],$$

the explicit opened coercion acts at p , and conversion transports the result back to $\tau'[y/x]$. This is the semantic step for which the extra premise of `S-PAIR-SINGLE` is needed.

The restriction still supports the intended abstract interface. If $\Gamma \vdash p :: P$, the checked examples derive

$$\begin{aligned} &\langle x:\{p\}; A:\{p\}..\{p\} \rangle \\ &<: \langle x:\{p\}; A:\perp..\{x\} \rangle \\ &<: \langle x:P; A:\perp..\{x\} \rangle. \end{aligned}$$

The first step changes an exact member into dependent abstract bounds while the first component is a singleton. The second step widens the first component. The order matters: after widening to an arbitrary P , the calculus does not generally permit changing the member.

This is a proof-motivated conservative restriction, not a demonstrated boundary of soundness. We have no counterexample to the unrestricted rule after singleton types and abstract selections have been separated.

4 Soundness

4.1 Statements

Because allocation extends the intrinsically scoped store, finite machine executions relate states at possibly different scope sizes. We write $\text{initial}(t) \longrightarrow^* s$ for this heterogeneous reflexive-transitive closure. Progress has its usual state formulation:

$$\text{Progress}(s) \stackrel{\text{def}}{\iff} \text{final}(s) \vee \exists s'. s \longrightarrow s'.$$

Theorem 4.1 (Finite-run type safety). *If $\emptyset \vdash t : T$ and $\text{initial}(t) \longrightarrow^* s$, then $\text{Progress}(s)$.*

The preservation invariant is more precise than source typing. Write $\Delta \Vdash_{\text{exact}} s : U$ when the store is recorded at exact value-introduction types and the current term and continuation are checked by the structural run-time judgments described below. Write $(\Gamma, T) \preceq (\Delta, U)$ when Δ extends Γ by zero or more allocations and U is weakened once for each extension.

Theorem 4.2 (Finite-run preservation). *If $\emptyset \vdash t : T$ and $\text{initial}(t) \longrightarrow^* s$, then there exist Δ and U such that*

$$(\emptyset, T) \preceq (\Delta, U) \quad \text{and} \quad \Delta \Vdash_{\text{exact}} s : U.$$

The source judgment is designed for source terms, not intermediate CK states. [Theorem 4.2](#) states closure of the actual machine invariant; [theorem 4.1](#) is its source-facing non-stuckness corollary.

4.2 Path resolution and exact stores

The first difficulty is path lookup. Under a well-typed exact store, every proper, precisely typed path resolves. Separately, resolution preserves ordinary source typing of its endpoint. It does *not* preserve the same precise synthesized type literally: a projection can synthesize a singleton, while the location it reaches synthesizes its context entry. The two

useful statements are therefore

$$\frac{\Gamma \vdash p :: T \quad \text{ExactStore}(\Gamma, \sigma)}{\exists x. \sigma \vdash p \Downarrow x}$$

$$\frac{\Gamma \vdash p : T \quad \text{ExactStore}(\Gamma, \sigma) \quad \sigma \vdash p \Downarrow x}{\Gamma \vdash x : T}$$

An exact store assigns a cell the syntax-directed type of the value that was allocated there. This differs from an ordinary store-typing relation that allows a cell to be recorded at a supertype. Exactness is needed for canonical forms: if a location is later viewed as a function, the proof must recover the actual introduction form and its original signature. In the allocation case, value inversion recovers the introduction type, and narrowing rechecks the suspended let body under that exact type.

4.3 Run-time conversion

Different paths may reach the same location. Let $p \equiv_\sigma q$ be the least congruence containing every pair of paths that co-resolve in σ . Types and generalized types admit conversion along this equivalence. Conversion is used, for example, to relate a selected member opened at $p.1$ to the same member opened at the location returned by the projection.

Ordinary weakening is not sufficient below binders: the new variable has no store cell and must not inherit accidental equations from the old scope. The proof therefore uses a scoped lift of $\equiv_\sigma$. Old equations are weakened, while the new variable is related only to itself.

The source rules embed into proof-only *structural* judgments for paths, subtyping, well-formedness, terms, continuations, and states. These judgments expose the constructors of the source derivation and add conversion by $\equiv_\sigma$. They do not change the source calculus; they provide the induction principles needed for preservation.

4.4 Generalized resolution and semantic maps

Source evaluation resolves term paths only to locations. The proof uses a generalized endpoint

$$e ::= \text{val}(x) \mid \text{type}(W),$$

so one resolution relation covers both term selection and stored type definitions. On term paths it agrees with source resolution.

The interpretation consists of three mutually inductive judgments: proper-type possibility, generalized endpoint realization, and semantic maps. A judgment $\text{Possible}_{\Gamma, \sigma}(x, T)$ says that location x can be used at proper type T . Its key cases are:

- a function type requires an actual stored abstraction and records the residual domain and codomain subtypings needed for beta reduction;
- a pair type requires an actual stored pair and realizers for its first component and opened member;

- $\{p\}$ requires p to resolve to x ; and
- $p.A$ requires $p.A$ to resolve to a stored type W , followed by a realizer of W .

Endpoint realization handles generalized types. A value endpoint realizes a proper type through Possible. A type endpoint W realizes $L..U$ by retaining a lower coercion $L \Rightarrow_s W$ and an upper coercion $W \Rightarrow_s U$.

The notation $\tau_1 \Rightarrow_s \tau_2$ denotes a finite, proof-relevant *semantic map*. Its constructors follow structural subtyping: identity, composition, conversion, top and bottom, singleton widening and aliasing, selection bounds, functions, the two pair rules, and intervals. It is a syntax of coercions rather than a semantic function, so the mutual definition remains strictly positive. Each code has an action on endpoint realizers:

$$\frac{\tau_1 \Rightarrow_s \tau_2 \quad e \in \llbracket \tau_1 \rrbracket_{\Gamma, \sigma}}{e \in \llbracket \tau_2 \rrbracket_{\Gamma, \sigma}}. \quad (\text{Map Action})$$

4.5 Fundamental lemma

The central induction is stated under simultaneous path substitution. Call ρ a *realized structural substitution* from Γ to (Δ, σ) when three conditions hold:

1. structural substitution transports every binding of Γ to its substituted type in Δ ;
2. ρ is a homomorphism from the source structural path relation to $\equiv_\sigma$; and
3. each source variable is mapped to a path that resolves to an endpoint realizing the substituted context entry.

The theorem has mutually proved path and subtyping parts.

Lemma 4.3 (Mapped substitution). *Suppose ρ is a realized structural substitution from Γ to (Δ, σ) .*

1. If $\Gamma \Vdash p :: \tau$, then there is an endpoint e such that $\rho(p)$ resolves to e and $e \in \llbracket \rho(\tau) \rrbracket_{\Delta, \sigma}$.
2. If $\Gamma \Vdash \tau_1 <: \tau_2$, then $\rho(\tau_1) \Rightarrow_s \rho(\tau_2)$.

Most cases follow the corresponding source constructor. Selection uses the maps stored in interval realization. Transitivity composes finite map codes. The singleton pair case uses co-resolution and the explicit opened premise, as described in section 3. The identity substitution and an exact store instantiate lemma 4.3 to obtain total generalized resolution and a semantic map for every structural subtyping derivation.

4.6 Canonical forms, preservation, and progress

Apply a semantic map to the realization of the location singleton $\{x\}$. If the target has function type, inversion of Possible produces an abstraction stored at x . It also retains the exact closure signature and the contravariant domain and covariant codomain residuals required by beta preservation. The corresponding pair result produces a stored pair and is used to resolve projections and selections.

Lemma 4.4 (Exact-store canonical forms). *Let σ be exact for Γ .*

1. *If $\Gamma \Vdash \{x\} <: (z : S) \rightarrow T$, then $\sigma(x)$ is an abstraction.*
2. *If $\Gamma \Vdash \{x\} <: \langle z : S; a : \tau \rangle$, then $\sigma(x)$ is a pair.*

Progress is then by induction on structural term and continuation typing. The application case uses the function part of lemma 4.4; path progress uses the pair part. Preservation proceeds by cases on the six machine transitions. Beta reduction uses the retained function-signature residuals. The let-allocation case extends the exact store and hence the intrinsic scope; all other cases preserve the current scope. Iteration gives theorem 4.2, and progress at its endpoint gives theorem 4.1.

5 Mechanization and scope

The development is checked with Lean 4 [2]. Syntax, contexts, stores, and states are indexed by scope size. Renaming, weakening, simultaneous path substitution, and opening are explicit. The default Lake target LambdaP contains the present calculus and its unconditional safety proof. The separate LambdaPHistory target retains the earlier calculus, failed proof invariants, and the checked source counterexample.

The source-facing theorems are:

Lean declaration	Result
<code>Tm.Ty.closed_progress</code>	initial-state progress
<code>Tm.Ty.closed_step_preservation</code>	one step from the initial state
<code>Tm.Ty.closed_finite_preservation</code>	finite-run preservation
<code>Tm.Ty.closed_finite_safety</code>	preservation and progress
<code>Tm.Ty.closed_type_safety</code>	finite-run non-stuckness

They are exported from `LambdaP/Safety.lean`. The imported default LambdaP target uses no `sorry`, `admit`, or user-declared axioms.

The main proof components can be located directly:

Declaration	Role
<code>Path.reduce_progress_precise</code>	source path progress
<code>Path.RuntimeEq</code>	co-resolution congruence
<code>Store.StructPreciseTy</code>	exact store invariant
<code>Path.StructCheck.mapped_subst</code>	path fundamental lemma
<code>Tau.StructSub.mapped_subst</code>	subtyping fundamental lemma
<code>mappedPreciseStructSafetyLaws</code>	canonical-form laws

The theorem does not assert termination: it quantifies over endpoints of finite executions. It does not establish algorithmic typing, type inference, or admissibility of transitivity. Path resolution is proved deterministic; machine-step determinism is not packaged as a theorem. Intermediate states are preserved by the exact structural invariant, not by a literal extension of the source term judgment.

6 Related work

The original DOT foundations isolate type members and path-dependent types in an object calculus [1]. Subsequent soundness proofs handle substantially richer combinations of recursive objects, refinements, and lattice structure [5]. The tight-typing proof of Rapoport et al. [3] separates reasoning

in general contexts from reasoning in restricted contexts where abstract members can be inverted. λ_p does not use inert contexts or tight typing. Its exact store and semantic maps are tailored to immutable pair chains and the much smaller source language.

pDOT extends DOT with paths of arbitrary length and singleton types, and gives a Coq-mechanized safety proof [4]. It is the closest point of comparison for the distinction between path equality and abstract selection. The present calculus differs in its dependent-pair representation of record chains, big-step missed-label lookup, and absence of recursive objects, intersections, and unions. The aim here is not to replace the pDOT metatheory, but to determine which proof obligations remain when paths and dependent members are separated from those features.

7 Conclusion

λ_p gives a small setting for paths through dependent pairs with abstract type members. The checked counterexample shows why singleton symmetry cannot treat membership in an abstract selection as path identity. This calculus makes the two roles syntactically separate. With that distinction and pair-member subtyping restricted to singleton first components, a direct proof establishes finite-run type safety. The open question left by this version is precise: whether the unrestricted dependent pair rule can be justified for the syntax-separated calculus, or whether it admits a counterexample not captured by the present semantic invariant.

References

- [1] Nada Amin, Tiark Rompf, and Martin Odersky. 2014. Foundations of Path-Dependent Types. In *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages and Applications*. Association for Computing Machinery, New York, NY, USA, 233–249. doi:10.1145/2660193.2660216
- [2] Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *Automated Deduction – CADE 28 (Lecture Notes in Computer Science, Vol. 12699)*, André Platzer and Geoff Sutcliffe (Eds.). Springer International Publishing, Cham, 625–635. doi:10.1007/978-3-030-79876-5_37
- [3] Marianna Rapoport, Ifaz Kabir, Paul He, and Ondřej Lhoták. 2017. A Simple Soundness Proof for Dependent Object Types. *Proceedings of the ACM on Programming Languages* 1, OOPSLA, Article 46 (2017), 27 pages. doi:10.1145/3133870
- [4] Marianna Rapoport and Ondřej Lhoták. 2019. A Path to DOT: Formalizing Fully Path-Dependent Types. *Proceedings of the ACM on Programming Languages* 3, OOPSLA, Article 145 (2019), 29 pages. doi:10.1145/3360571
- [5] Tiark Rompf and Nada Amin. 2016. Type Soundness for Dependent Object Types (DOT). In *Proceedings of the 2016 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications*. Association for Computing Machinery, New York, NY, USA, 624–641. doi:10.1145/2983990.2984008